

COMPUTER SCIENCE MENTORS 61A

December 2 – December 5, 2025

1 SQL

1. CS 61A wants to start a fish hatchery, and we need your help to analyze the data we've collected for the fish populations! Running a hatchery is expensive—we'd like to make some money on the side by selling some seafood (only older fish of course) to make delicious sushi.

The table `fish` contains a subset of the data that has been collected. The SQL column names are listed in brackets.

`fish`

Species [species]	Population [pop]	Breeding Rate [rate]	\$/piece [price]	# of pieces per fish [pieces]
Salmon	500	3.3	4	30
Eel	100	1.3	4	15
Yellowtail	700	2.0	3	30
Tuna	600	1.1	3	20

Write a query to find the total number of fish in the ocean. Additionally, include the number of species we summed. Your output should have the total number of species in the first column and the total population in the second column. It should yield in a table with a single row.

```
SELECT COUNT(species), SUM(pop)
FROM fish;
```

Write a query to find the three most populated fish species.

```
SELECT species
FROM fish
ORDER BY pop DESC
LIMIT 3;
```

Profit is good, but more profit is better. Write a query to select the species that yields the greatest number of pieces for each price point. For example, if "Trout" and "Catfish" both cost 100 dollars/piece, you should output the species that has the greater number of pieces. Your output should include the species, price, and number of pieces per fish.

```
SELECT species, price, MAX(pieces)
FROM fish
GROUP BY price;
```

Alternate solution

```
SELECT species, price, pieces
FROM fish
ORDER BY pieces / price DESC
LIMIT 2;
```

2. After more than 100 years of operation, the Ringling Bros. circus is closing. A victory for animal rights advocates, the circus' closure poses a challenge for the zoologists tasked with moving the circus' animals to more suitable habitats.

The zoologists must first take the animals in a freight elevator with a weight limit of 2000. In order to speed up the process, the zoologists prefer to take groups of animals of the same species in the elevator, rather than one animal at a time.

Assume the zoologists will only put all of the animals of a particular species in the elevator, or take animals of that particular species one at a time.

You have access to the table `animals`, with columns containing the animals' names, heights, weights, and species.

Name	Height	Weight	Species
Wilbur	4.1	150	pig
Tigress	4.4	700	tiger
Phil	3.3	79	pig
Dug	3.5	40	dog
Buddy	4	51	dog
Marty	4.9	300	zebra
Richard Parker	5.2	918	tiger

Write a query that returns the collective weight and species of animals in a group where there is more than one animal of a particular species in a group, and the collective weight of the animals in the group is less than 2000.

Your query should yield the following result.

```
91 dog
229 pig
1618 tiger
```

```
SELECT SUM(weight), species FROM animals
GROUP BY species HAVING COUNT(*) > 1 and SUM(weight) < 2000;
```

To take the animals to their new habitats, the zoologists load the animals into trucks. The zoologists would like to reduce the number of trips taken by pairing up the animals for the trip (species does not matter). However, the trucks have two restrictions:

1. The maximum height of any animal is 5.0.
2. The total weight of both animals cannot exceed 300.

Write a query which will yield all possible pairings in a table (which is basically the table below). To match the table below, make sure that the left column animals' names come earlier in the alphabet than their partners.

Buddy	Dug
Buddy	Phil
Buddy	Wilbur
Dug	Phil
Dug	Wilbur
Phil	Wilbur

```
SELECT a.name, b.name
FROM animals AS a, animals AS b
WHERE a.name < b.name AND a.weight + b.weight <= 300 AND a.height <= 5
AND b.height <= 5;
```

3. CSM 61A Content Team wants to put together an end of the year party for all its mentors (and some special guests, too!) themed around everyone's favorite things! Examine the table, `mentors`, depicted below, and answer the following questions.

Name	Food	Color	Editor	Language	Animal
Alyssa	Pork Bulgogi	Navy Blue	Vim	Java	Sea Otter
Vibha	Pasta	Teal	VSCode	Java	Naked Mole Rat
Adit	Protein Bar	Black	Vim	Python	Gorilla
Esther	Goldfish	Pastel Pink	VSCode	Python	Bunny
Aiko	Fries	Sky Blue	VSCode	Java	Cat
Aurelia	Dumpling	Pastel Yellow	VSCode	Python	Panda
Kaelyn	Popcorn	Blue	VSCode	Java	Panda

Write a query that has the same data, but alphabetizes the rows by name. (Hint: Use `order by`.)

```
Adit|Protein Bar|Black|Vim|Python|Gorilla
Aiko|Fries|Sky Blue|VSCode|Java|Cat
Alyssa|Pork Bulgogi|Navy Blue|Vim|Java|Sea Otter
Aurelia|Dumpling|Pastel Yellow|VSCode|Python|Panda
Esther|Goldfish|Pastel Pink|VSCode|Python|Bunny
Kaelyn|Popcorn|Blue|VSCode|Java|Panda
Vibha|Pasta|Teal|VSCode|Java|Naked Mole Rat
```

```
SELECT *
FROM mentors
ORDER BY name;
```

Write a query that lists the food and the color of every person whose favorite language is *not* Python.

```
Pork Bulgogi|Navy Blue
Pasta|Teal
Fries|Sky Blue
Popcorn|Blue
```

```
SELECT food, color
FROM mentors
WHERE language != 'Python';

-- With aliasing (treating the table as a Python object) --
SELECT m.food, m.color
FROM mentors as m
WHERE m.language <> 'Python';
```

Write a query that lists all the mentors along with their favorite food if their favorite color is green.

Matthew|Pie

Ivan|Ramen

```
select name, food
  from mentors
 where color = 'Green';

-- With aliasing
select m.name, m.food
  from mentors as m
 where m.color = 'Green';
```

Write a query that lists all the pairs of mentors who like the same language. (How can we make sure to remove duplicates?)

Catherine|Jamie

Ethan|Kenny

```
select m1.name, m2.name
  from mentors as m1, mentors as m2
 where m1.language = m2.language and m1.name < m2.name;
```

4. In this question, you have access to two tables.

Grades, which contains three columns: `day`, `class`, and `score`. Each row represents the score you got on a midterm for some `class` that you took on some day.

Outfits, which contains two columns: `day` and `color`. Each row represents the color of the shirt you wore on some day. Assume you have a row for each possible day.

grades

Day	Class	Score
10/31	Music 70	88
9/20	Math 1A	72

You want to find out how you did on the midterms to which you wore your blue shirt to. So write a query which is going to yield a table where there is a row for each day you wore a blue shirt to a midterm (or multiple midterms) and then sum of midterm scores for that day.

```
SELECT grades.day, SUM(score)
FROM grades, outfits
WHERE outfits.color = 'blue' and outfits.day = grades.day
GROUP BY grades.day;
```

You want to find out which classes you need to prepare for the most by determining how many points you have so far. However, you only want to do so for classes where you did relatively poorly.

Write a query that will output the sum of your midterm scores for each class along with the corresponding class, but only for classes in which you scored less than 80 points on at least one midterm. List the output from highest to lowest total score.

```
SELECT SUM(score), class
FROM grades GROUP BY class
HAVING MIN(score) < 80 ORDER BY SUM(score) DESC;
```

2 Trees

1. The total weight of a tree is defined as the sum of the labels of all its nodes. A tree is defined to be `equally_weighted` if the total weight of each of its branches are equal. A leaf is assumed to be `equally_weighted`.

Complete the following functions for `equally_weighted` and `num_eq_weight`. You may use the below definition of the `total_weight` function in your answer.

```
def total_weight(t):
    '''
    Return the total weight of a tree, i.e. the sum of all its labels.
    >>>total_weight(Tree(1, [Tree(2), Tree(3,[Tree(4)])]))
    10
    '''
    weight = t.label + sum([total_weight(branch) for branch in t.branches])
    return weight
```

```

(a) def equally_weighted(t):
    '''
    Return whether a tree is equally weighted.
    >>>equally_weighted(Tree(1))
    True
    >>>equally_weighted(Tree(1,[Tree(2), Tree(1, [Tree(1)])]))
    True
    >>>equally_weighted(Tree(0, [Tree(3), Tree(2, [Tree(3)])]))
    False
    '''
    _____ = [_____]

    for _____ in _____:

        if _____:

            return _____

    return _____

def equally_weighted(t):
    all_weights = [total_weight(b) for b in t.branches]
    for weight in all_weights[1:]:
        if weight != all_weights[0]:
            return False
    return True

```

(b) **Note:** You are allowed to use `equally_weighted` in this part.

```
def num_eq_weight(t):
    '''
    Return the number of equally weighted subtrees of t. Note that t is
    considered a subtree of itself.
    >>> num_eq_weight(Tree(1, [Tree(4), Tree(3,[Tree(1)])]))
    4
    >>> num_eq_weight(Tree(1, [Tree(9),
                                Tree(1, [Tree(4),
                                           Tree(3,[Tree(1)])])]))
    6
    >>> num_eq_weight(Tree(1, [Tree(8, [Tree(1)]),
                                Tree(1, [Tree(4),
                                           Tree(3,[Tree(1)])])]))
    7
    '''
    val = _____

    if _____:

        return _____
    else:
        return val

def num_eq_weight(t):
    val = sum([num_eq_weight(b) for b in t.branches])
    if equally_weighted(t):
        return 1 + val
    else:
        return val
```